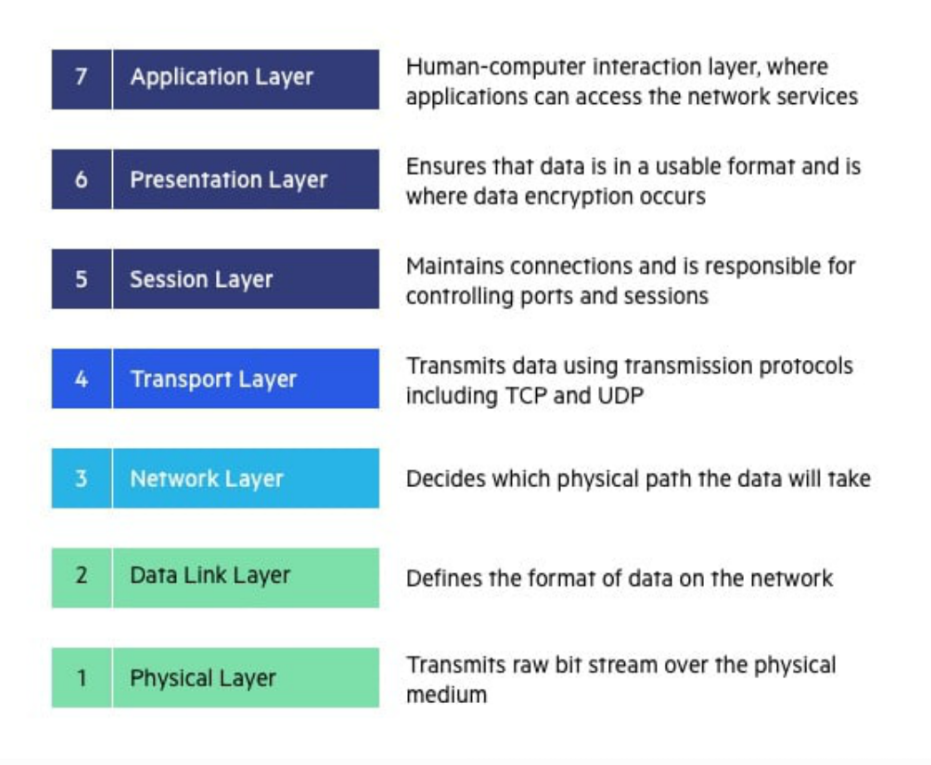
==========network protocols and high-level design(02-March-2025):

* **Network Protocols Defined**: They are a set of rules that govern how two systems or computers communicate over a network.



* **OSI Model**: The video mentions the seven layers of the OSI model, focusing on the **Application Layer** and **Transport Layer**.
* **Application Layer Protocols**:
  * **Client-Server Protocols**:
    * **HTTP**: Used for accessing web pages.
    * **FTP**: Used for file transfer, maintaining separate control and data connections.
    * **SMTP**: Used for sending emails, often in conjunction with IMAP or POP for receiving emails.
    ***WebSocket
  * **Peer-to-Peer Protocol**:
    * **WebRTC**
* **Client-Server Model**: Involves a client making requests and a server responding. Communication is unidirectional, initiated by the client.
* **WebSocket Protocol**: Enables bidirectional communication between client and server, crucial for real-time applications like messaging apps.
* **HTTP in Detail**: It is connection-oriented, used to access web pages and navigate between them.
* **SMTP, IMAP, and POP**: SMTP is for sending emails, while IMAP is preferred over POP for receiving as it allows reading emails from multiple devices.
* **Peer-to-peer model**: Clients can talk to each other without the need of a server.
* **Transport Layer Protocols**:
  * **TCP/IP**: Establishes a virtual connection, divides data into sequenced packets, ensures reliability through acknowledgments and retransmissions.
  * **UDP**: Does not guarantee order or reliability, but it's faster. It's suitable for live streaming and video calling.
* **When to Use Which Protocol**
  * **WebSocket**: For messaging apps requiring bidirectional communication.
  * **UDP**: For live streaming and video calling where speed is crucial, and some data loss is acceptable.
  * **HTTP**: For standard web-based interactions.
  * **FTP**: The video mentions that FTP is not secure because the data connection is not encrypted.

CAP Theorem

The CAP Theorem defines desirable properties for distributed systems with replicated data.
CAP stands for Consistency, Availability, and Partition Tolerance.

Consistency: After a successful write on any node, all reads from any node should return the same data.
Availability: All nodes should respond to requests.
Partition Tolerance: The system should continue to operate even if communication between nodes is broken.

A key constraint of the CAP Theorem is that you can only choose two out of the three properties in a distributed system.

You can choose CA, CP, or AP, but not all three (CAP).

Why not all three?

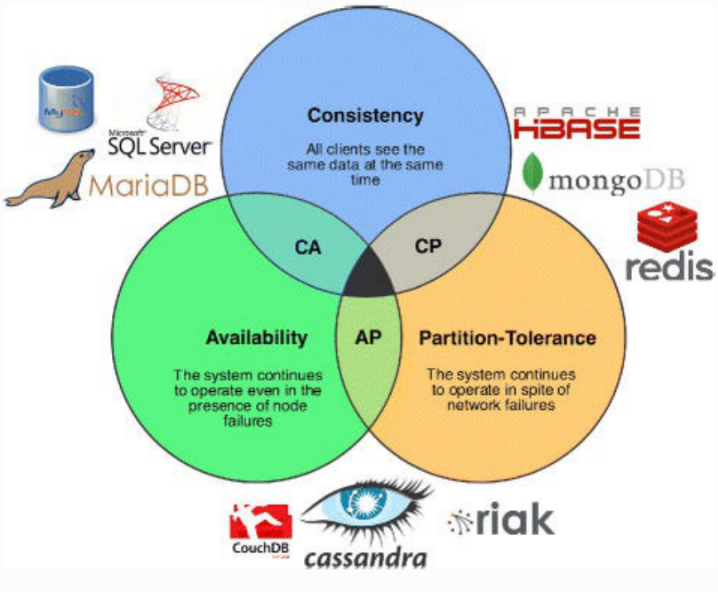
If a partition occurs (communication breaks down), you have to choose between Consistency and Availability.
If you choose consistency, you might have to shut down a node, sacrificing availability.
If you choose availability, you might return stale data from some nodes, sacrificing consistency.
If you choose both Consistency and Availability, your system will not be Partition Tolerant.

Common Scenarios

CP (Consistency and Partition Tolerance): If a partition occurs, the system will prioritize consistency and might become unavailable.
AP (Availability and Partition Tolerance): If a partition occurs, the system will prioritize availability and might return inconsistent data.
CA (Consistency and Availability): This is not partition-tolerant. If a partition occurs, the system might have to shut down.

Practical Considerations

In modern distributed systems, **partition tolerance is often a must-have.**
Therefore, the choice often comes down to choosing between consistency (CP) and availability (AP).



Microservices

The video is the third part of a series on High-Level Design (HLD), focusing on the critical topic of Monolithic versus Microservices architectures.
This topic is highly relevant for HLD interviews, with potentially 50 questions arising from microservices and their design patterns.

II. **Monolithic Architecture**
Definition: In a monolithic architecture, all functionalities of an application are contained within a single application.

Example: An online store where all functions—order generation, inventory management, login, billing, and payments—are within one application.

Disadvantages:
Overload IDE: Large monolithic applications can be challenging to load and work with in an Integrated Development Environment (IDE).

Integrated Development Environment (IDE) : A software application that provides comprehensive facilities to computer programmers for software development.

Hard Scaling: Scaling is difficult because all functionalities are within one application. Changes, even minor ones, require redeployment of the entire application.
Scaling: The ability of a system to handle increased load

Tight Coupling: Components are tightly coupled, meaning a change in one part can impact many others, necessitating extensive testing.
Tight Coupling: Components of a system are highly dependent on each other.

Inefficient Scaling: Scaling specific functionalities is impossible; the entire application must be scaled, leading to resource inefficiency.
Debugging Challenges: Debugging is complex due to the large codebase.

III. Microservices Architecture

Definition : Microservices architecture involves breaking down a large application into smaller, independent services.

Advantages:
Easy Management: Managing smaller components is simpler for developers.
Scalability: Each service can be scaled independently based on need.
Cost-Effective: Resources can be allocated efficiently, scaling only necessary components.

Disadvantages:
Proper Decomposition: Breaking down the monolithic service into microservices must be done correctly to ensure loose coupling.
Loose Coupling: Components of a system have minimal dependencies on each other.
Increased Latency: Improper decomposition can lead to increased communication between services, raising latency.
Latency: The delay in data transfer between two points.

Deployment Complexity: Deploying changes requires careful monitoring of dependent services to avoid breaking them.
Debugging Challenges: Identifying the root cause of errors can be difficult as it may span multiple services.
Transaction Management: Managing transactions across multiple databases (one for each service) is complex.
Transaction Management: Ensuring that database operations are performed reliably and consistently.

== Microservices Design Patterns

Microservices can be divided into 'micro' and 'services', raising the **question of how small a "micro" should be.**

Different Phases of Microservices:

Decomposition
Database
Communication
Integration
Observability

Each phase involves different patterns that can be mixed and matched to create a microservice.

*Decomposition Patterns:

Decomposition patterns guide how to break down a large application into smaller services.Two Main Patterns:

Decompose by Business Capability: Divide services based on business functions (e.g., order management, product management).*Requires good knowledge of business functions.*

Decompose by Subdomain (Domain-Driven Design - DDD): Divide based on subdomains within a business domain (e.g., order tracking within order management).
A domain can have multiple microservices.

Microservices Design Patterns Part 2

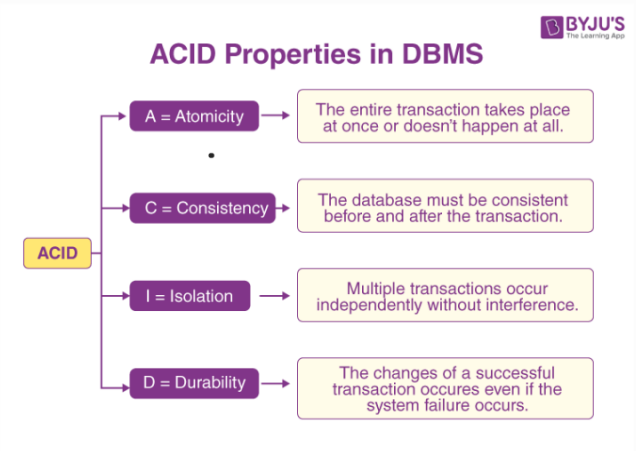
Three important microservices design patterns: **Strangler Pattern**, **Saga Pattern**, and **CQRS**.

Strangler Pattern

This pattern is used when **refactoring a monolithic service into microservices**.
It involves gradually migrating functionality from the monolith to microservices, routing a percentage of traffic to the new services.
A controller manages traffic, allowing you to test and increase the load on microservices incrementally.
This approach minimizes risk by allowing quick rollbacks if issues arise in the new microservices.

*** Data Management in Microservices**

while loading the data we must follow ACID properties:



There are two main approaches:
Database per service: Each microservice has its own dedicated database.
Shared database: Multiple microservices share a single database.

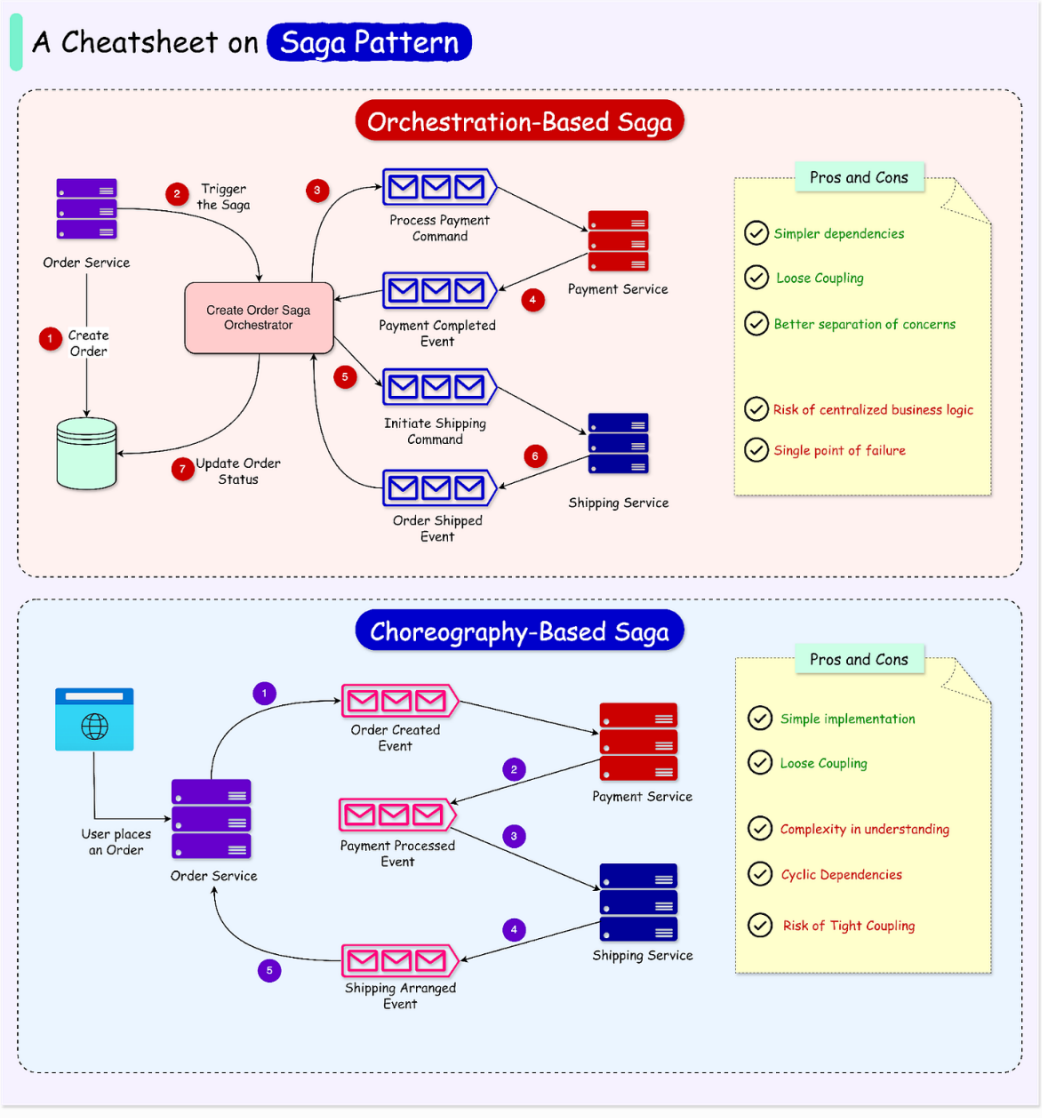
The video explains why "database per service" is generally preferred over a shared database:
However, "database per service" introduces challenges in maintaining data consistency across services.

Scalability: Each service can scale its database independently.
Independence: Teams can modify their service's database without affecting others.
Technology diversity: Services can choose the database technology that best fits their needs.

The Saga pattern addresses the challenge of maintaining data consistency across multiple databases in a microservices architecture.
It manages transactions that span multiple services by using a sequence of local transactions.
If one transaction fails, the Saga pattern executes compensating transactions to undo the changes made by previous transactions, ensuring data consistency.

There are two main implementations of the Saga pattern:

Choreography: Each service listens for events and acts accordingly.
Orchestration: A central orchestrator manages the Saga and tells each service when to execute its local transaction.



CQRS (Command Query Responsibility Segregation)

CQRS separates the operations that modify data (commands) from the operations that read data (queries).
In a microservices architecture, this often means having separate data models for reads and writes.
Write operations are handled by one set of databases, while read operations are handled by a separate database optimized for queries.
Data is kept consistent between the write and read databases through events or other synchronization mechanisms.
CQRS can improve performance and scalability, especially for applications with a high read-to-write ratio.

Keywords Explanation:
Monolithic Service: A single, large application where all components are tightly coupled.
Microservices: An architectural approach where an application is structured as a collection of small, independent services.
Refactoring: The process of restructuring existing computer code without changing its external behavior.
Decomposition Pattern: Strategies for breaking down a monolithic application into microservices.
Traffic Routing: Directing network traffic to different services based on certain criteria.
Rollback: Reverting a system or database to a previous state, typically to recover from an error.
Data Consistency: Ensuring that data remains accurate and up-to-date across all parts of a system.
Local Transaction: A transaction that involves only a single database.
Distributed Transaction: A transaction that involves multiple databases or systems.
Eventual Consistency: A consistency model where data will become consistent across all systems eventually, but there may be a delay.
Event Sourcing: A design pattern where all changes to an application's state are stored as a sequence of events.
Scalability: The ability of a system to handle an increasing amount of work by adding resources.
ACID Properties: A set of properties that guarantee database transactions are processed reliably. ACID stands for Atomicity, Consistency, Isolation, and Durability.

Zero to a Million Users

- Introduction

  - The video discusses scaling from zero to one million users, covering concepts like CDN, horizontal and vertical scaling, load balancer, and database.

1- **Single Server**

  - Initially, a basic project starts with a single server hosting the application and database.
  - The client directly interacts with this server.

2- **Application and DB Server Separation**

  - To scale, the application and database are separated into different layers.
  - This allows independent scaling of both.

3- **Load Balancer and Multiple App Servers**

  - To handle more traffic, a load balancer is introduced with multiple application servers.
  - The load balancer distributes traffic and enhances security.
  - Multiple servers are needed to handle the limited requests an application server can handle.

4- **Database Replication**

  - Database replication using master-slave setup is implemented.
  - Write requests go to the master, while read requests are handled by slaves.
  - This setup ensures redundancy and handles failures.

5- **Use of Cache**

  - A cache is used to improve performance by storing frequently accessed data.
  - DB Operations are very expensive
  - The application first checks the cache before querying the database.
  - This reduces the load on the database.

6 - **CDN (Content Delivery Network)**

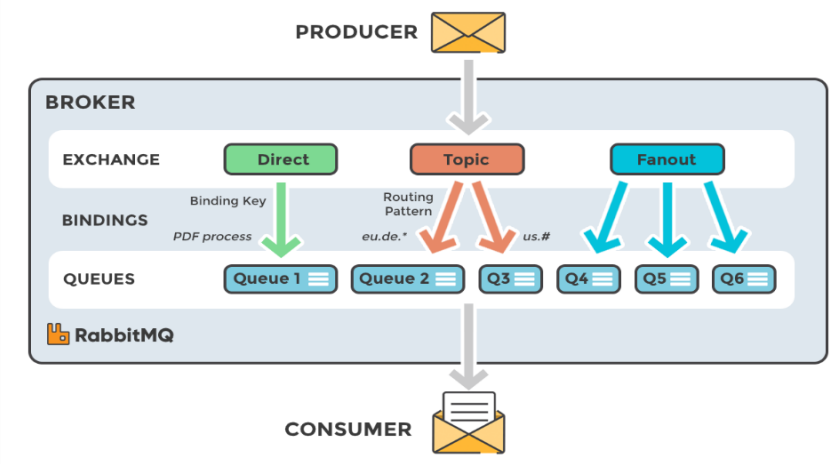
- CDNs are used to cache and deliver static content to users based on their location.
- CDNs improve performance and reduce latency for global users.
- They also enhance security by handling attacks before they reach the origin server.

7- Multiple Data Centers

- Multiple data centers are set up to distribute load and ensure high availability.
- Load balancers route traffic to the nearest data center.
- This setup minimizes latency and handles data center failures.

8- **Messaging Queue**

- Messaging queues are used to handle asynchronous tasks.
- They decouple services and ensure reliable processing of tasks like sending notifications.
- Technologies like **RabbitMQ** and **Kafka** are used.
- The video explains concepts like **exchange**, **binding** and **routing** key.



9- **Database Scaling**

- Database scaling is done in two ways: vertical and horizontal.
- Vertical scaling involves increasing the capacity of existing servers.
- Horizontal scaling involves adding more database nodes.
- Horizontal scaling is implemented through sharding, which can be horizontal or vertical.
- **Horizontal sharding divides rows into multiple tables.**
- **Vertical sharding divides columns**.
- Sharding drawbacks and solutions are also discussed.

Keywords

- CDN (Content Delivery Network): A geographically distributed network of servers that cache static content, delivering it to users based on their location to reduce latency and improve load times.
- Load Balancer: A device or software that distributes network traffic across multiple servers to ensure no single server is overwhelmed, improving responsiveness and availability.
- Database Replication: The process of copying data from a primary database (master) to one or more secondary databases (slaves) to provide redundancy and improve read performance.
- Cache: A high-speed data storage layer that stores frequently accessed data to reduce the need to fetch it from slower storage mediums, improving application performance.
- Messaging Queue: A system that allows applications or microservices to communicate asynchronously by sending and receiving messages, decoupling the sender and receiver and ensuring reliable message delivery.
- Vertical Scaling: Increasing the resources (CPU, RAM, storage) of a single server to handle increased load.
- Horizontal Scaling: Adding more servers to a system to distribute the load and increase capacity.
- Sharding: A database partitioning technique that divides a large database into smaller, more manageable pieces (shards) that can be distributed across multiple servers.

Consistent Hashing

Consistent hashing helps in sharding and is useful in various scenarios.
What is Hashing? Hashing involves a hash function that converts an arbitrary length input (key) into a fixed-length value.
Hash Function: A function that takes an input of any size and produces a fixed-size output.
Arbitrary Length Input: Data of any size.
Fixed Length Output: A value of a specific, predetermined size.
Mod Hashing: A technique used to map the hash value to an index within a hash table of a fixed size.

Problem with Traditional Hashing:
Traditional hashing works well with a fixed-size hash table .
Problems arise when the size of the hash table changes (e.g., in load balancing or distributed databases)
Load Balancing: Distributing network traffic across multiple servers to ensure no single server is overwhelmed.
Distributed Databases: Databases where data is stored across multiple servers.

Example Scenario:
In load balancing, requests need to be distributed across application servers
In distributed databases, data needs to be divided among multiple database servers
Using mod hashing, if a server is added or removed, the modulus changes, leading to a need to rebalance all the data
Rebalancing: Redistributing data across servers when the number of servers changes.
This rebalancing is inefficient, especially with millions of database entries

Goal of Consistent Hashing:
Minimize the amount of data that needs to be rebalanced when servers are added or removed
Ideally, only a small percentage of keys should need to be remapped (1/N, where N is the number of nodes)

Consistent Hashing Process
Virtual Ring: Consistent hashing uses a virtual ring. A conceptual ring structure where hash values are mapped.

Servers are placed on this ring using a hash function
Keys are also placed on the ring using the same hash function
To determine which server handles a key, you move clockwise around the ring until you find a server

Adding a Server: When a server is added, only the keys that would have been handled by the next server in the ring need to be remapped.
Removing a Server: When a server is removed, the keys it handled are taken over by the next server in the ring.

Addressing Uneven Distribution

A disadvantage of basic consistent hashing is that servers might not be evenly distributed on the ring, leading to uneven load. V**irtual Node**s (Objects): To solve this, each physical server can be represented by multiple virtual nodes on the ring.
Virtual Nodes: Multiple points on the ring that represent the same physical server.
This helps distribute the load more evenly.
The number of virtual nodes is chosen to achieve a balance between load distribution and overhead

Summary
Consistent hashing minimizes rebalancing when the number of servers changes.
It uses a virtual ring to map servers and keys. Virtual nodes help distribute load evenly.
Consistent hashing is useful in dynamic environments like load balancing and distributed databases.

Back-of-the-Envelope Estimation(create own sheet)

It's a crucial skill for high-level design questions in interviews and practice.
Helps to make informed decisions about system design by considering constraints.
Drives the decision for system design.

Key Considerations:
The estimations are rough and high-level, not exact figures.
It's advised **not to spend too much ti**me on it.
Keep the assumption values simple for easy computation.

Cheat Sheet and Basic Assumptions:
The video provides a cheat sheet for **traffic and storage calculations, using multiples of three zeros.**
It explains the conversions between (3)KB, (6)MB, (9)GB, (12)TB, and (15)PB.
Basic assumptions for data types are provided: 2 bytes for a character, 8 bytes for long/double, and 300 KB for an average image, 1 byte for ascii , 2 for unicode.

Core Computations

Generally, three main things are computed: **the number of servers needed, RAM, and storage capacity.**
The trade-off based on the CAP theorem is also defined.

Facebook Estimation: **Traffic Estimation**

The estimation begins with traffic, assuming **1 billion total users**.
Daily active users are estimated at **25% of the total, which is 250 million**.
Each user is assumed to perform **five read and two write** operations.(250x7)
The per-second traffic is calculated to be **18,000 queries per second.**

Facebook Estimation: **Storage Assumption**

Each user makes **two posts daily,** with each post containing **250 characters**.(250x2 byte=500x2 post=1TB)
10% of users upload one image, with **each image being 300 KB**.(100 million x300 =30 TB)
Calculations show a need for 250 GB per day for posts and 8 TB per day for images.
For 5 years of data, the storage capacity is estimated at 500 TB for posts and 16 PB for images.

Facebook Estimation: **RAM Estimation**

The assumption is made to cache the **last five posts for each user**.
This leads to a calculation of 750 GB of memory needed.
With each machine holding 75 GB, 10 machines are needed for caching.
The estimated latency is 500 milliseconds 95% of the time.
With one server handling 100 requests per second, 180 servers are needed.

Summary of Facebook Estimation:
The estimation concludes with 180 servers, 750 GB RAM, and storage capacities of 500 TB for posts and 16 PB for images.

Trade-off Based on CAP Theorem:
The video discusses the CAP theorem and the trade-off between consistency, availability, and partition tolerance.
For Facebook, the choice is to prioritize availability and partition tolerance over consistency.

Differences between SQL and NoSQL databases

Differences between SQL and NoSQL databases, covering their structure, nature, scalability, and properties, to help you decide which one to use.

Here's a breakdown:

Introduction

The video addresses the common interview question of choosing between SQL and NoSQL databases .
It emphasizes the importance of providing a rationale for your choice .
The scope includes when to use SQL vs. NoSQL and basic information about each .
It excludes in-depth SQL and NoSQL tutorials.

SQL

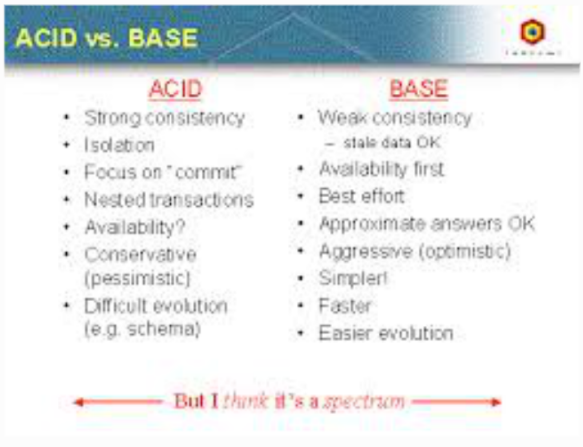
Structure: SQL databases use a **structured query language** to manage relational database management systems (RDBMS) Data is stored in tables with rows and columns, with **predefined schemas and relations between tables.**
Nature: SQL databases are generally **concentrated,** meaning all data for a particular entry is stored on a **single server**
Scalability: Vertical scaling (increasing RAM, storage) is preferred in SQL. Horizontal scaling using sharding is possible but not as well-supported.
Property: SQL databases follow ACID properties (Atomicity, Consistency, Isolation, Durability), ensuring data integrity.

NoSQL(Not only SQL)

Structure: N**oSQL databases work with unstructured data**, including **key-value**, **document**, **column-wise**, and g**raph databases.**

Key-value: Simple, fast, queries based on keys.(Dynomo DB)
Document: Stores JSON or XML, allows querying on values.(MongoDB)
Column-wise: Data stored in columns, dynamic number of columns .
Graph: Uses nodes and edges to represent relationships, suitable for social networking and recommendation engines.

Nature: NoSQL databases are distributed, with data spread across multiple nodes.
Scalability: Horizontal scaling (adding more nodes) is preferred.
Property: **NoSQL databases follow BASE properties (**Basically Available, Safe State, Eventual Consistency).



W**hen to Use Which**

SQL: Use when you need flexible query functionality, relational data, and strict data integrity (ACID).
NoSQL: Use when you need basic query search, non-relational data, high availability, high performance, a**nd can tolerate some inconsistency (BASE).**

Designing a rate limiter:

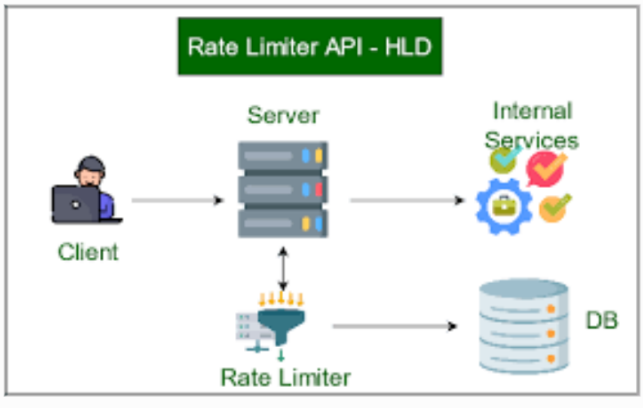
The video discusses rate limiting, which is preventing (DDOS) attackers from overwhelming a server with requests. Rate limiting is crucial to avoid server downtime and ensure genuine users can access the service.

The video covers **five key algorithms for rate limiting:**

Token Bucket: This algorithm uses a bucket that holds tokens, representing the number of allowed requests. Each request consumes a token, and tokens are added back to the bucket over time. If the bucket is empty, requests are denied(429 issue).
Leaking Bucket: This algorithm processes requests at a constant rate, like water leaking from a bucket. Incoming requests are added to the bucket, and if the bucket is full, excess requests are dropped.(429 issue).
F**ixed Window Counter**: This algorithm divides time into fixed windows and counts the number of requests within each window. Once the count exceeds a threshold, further requests are blocked until the next window.
Sliding Window Log: This algorithm maintains a log of request timestamps within a sliding window. It determines if a request is allowed by checking the number of requests within the current window.
Sliding Window Counter: This algorithm combines aspects of the fixed window counter and sliding window log. It estimates the number of requests in the current sliding window by considering the previous window's count and the current window's count.

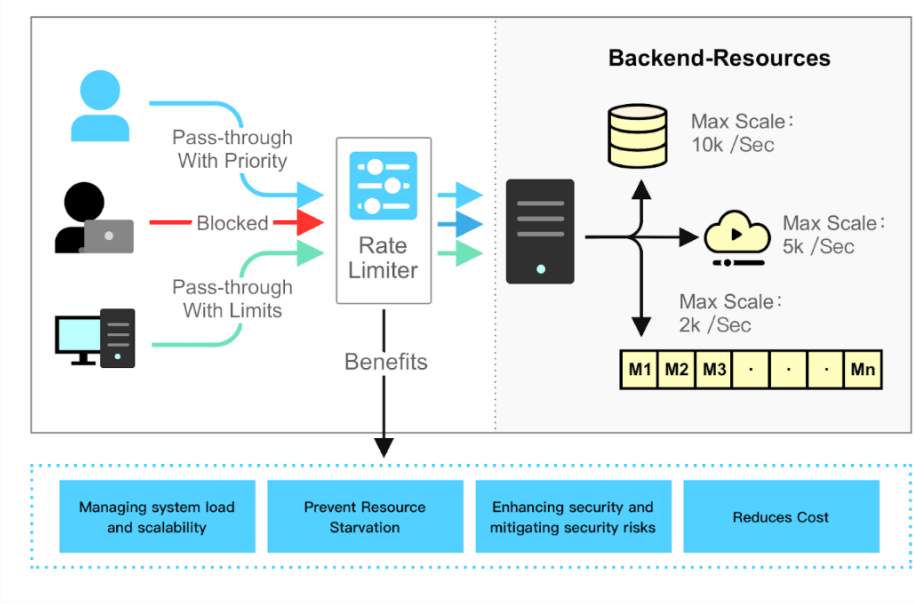
Designing a Rate Limiter:
the components needed for a rate limiter, including:

Counter: To track request counts.(radies)
Algorithm: To implement the chosen rate-limiting strategy.
Configuration: To define rate limits (e.g., requests per minute).
Cache: To store configuration files for quick access.



A typical setup involves a client sending requests to a rate limiter, which then forwards them to the server if within the defined limits. For distributed systems, a centralized data store like **Redis can be used to maintain atomicity(not good) and consistency across multiple rate limiters.**

When a request is denied due to rate limiting, the HTTP status code 429 (Too Many Requests) is typically returned.

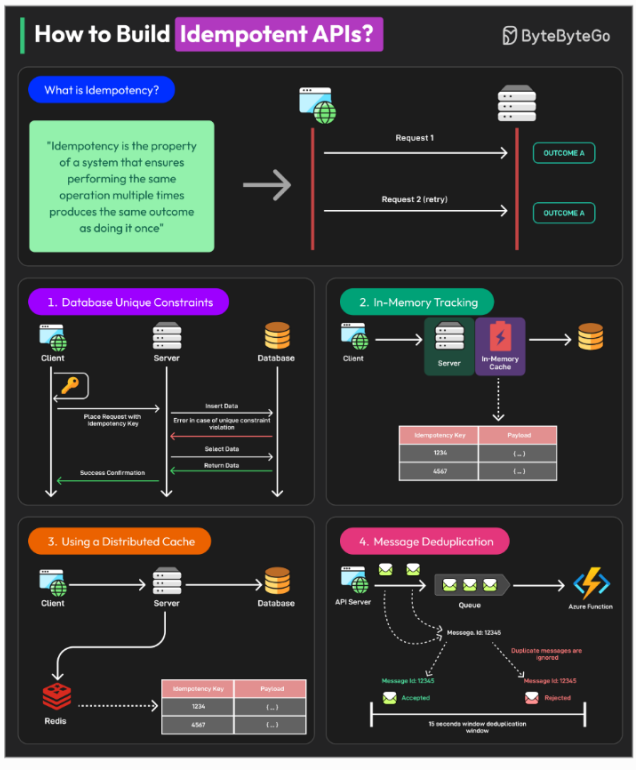


Idempotency

Idempotency vs. Concurrency
Concurrency: Multiple users accessing the same resource.
Idempotency: Safely retrying requests without unintended side effects.Enables clients to retry requests safely, without causing unintended side effects. For example, adding an item to a cart should only add one item, even with multiple requests.

HTTP Methods and Idempotency:
GET , **DELETE** and **PUT** are idempotent by nature. Multiple identical requests have the same **effect as a single request**.
POST is not idempotent by nature. Each request can create a new resource. example: Payments

Problems Leading to Duplicate Requests:
Sequential: Client timeout, leading to retries while the server is still processing.
Parallel: Duplicate requests arriving at the server at the same time.



Approach to Idempotency:
Client and server agree on using a unique ID (**idempotency key)** for each operation.
The client generates the idempotency key and includes it in the request header .

Detailed Flow:

1.Client generates a new and unique idempotency key .
2.Client sets the key in the request header and calls the server.
3.Server validates if the idempotency key is present in the header . Server checks if the idempotency key already exists in the database.
4.If the key is new, the server processes the request and saves the key with a 'created' status .
5.Upon successful operation, the server updates the key status to 'consumed'.
For duplicate requests, the server checks the key's status:

'Consumed': Returns HTTP 200, indicating the request was already completed .
'Created'(201): Returns HTTP 409, indicating a conflict as a previous request is still processing.

Handling Parallel Requests
Use a critical section (e.g., **mutex,sync.**) to ensure only one request processes the idempotency key at a time

Handling Multiple Servers
Use a cache for faster synchronization across different servers and databases.

How to design a high availability and resilient system?

Introduction

1. Designing a high availability architecture, also known as data resilience architecture
2. The goal is to achieve 99.999% availability and avoid a single point of failure
3. Active-passive and active-active architectures

Basic Single Node Architecture

A client sends a request to a load balancer, which directs it to an application layer (microservices) .
The application connects to a primary database (DB) to store data
If the DB goes down, the entire application fails, resulting in a single point of failure
This architecture does not provide 99.999% availability or data resilience

Multi-Node Architecture: **Active-Passive**

Involves two or more data centers, e.g., Data Center 1 (Mumbai) and Data Center 2 (Pune) .
Only one data center is primary (live, read-write), while others are replicas (disaster recovery) .
Requests to the primary data center are processed directly.
Requests to the disaster recovery data center are forwarded to the primary DB for write operations .
Data is synced from the primary DB to replicas .
Oracle, MySQL, and PostgreSQL are not multi-master, meaning they can only write to one primary DB.
Read requests can be served by the replica DBs.
If the primary DB fails, traffic is routed to the disaster recovery data center, which is then made primary.

Disadvantages of **Active-Passive**

Latency add-on: Requests to the disaster recovery data center may experience higher latency
Downtime during failover: There is a delay while the disaster recovery data center is made primary

Multi-Node Architecture: **Active-Active**

Uses databases like Cassandra and NoSQL that support multi-master, allowing multiple live DBs .
Each data center has its own primary DB, and there is bi-directional synchronization between them .
Requests can be served by any data center
Synchronization is complex, with potential conflicts if changes happen simultaneously in both data centers.

Advantages of **Active-Active**

Full utilization of resources in all data centers.
Higher traffic handling capacity.

Active-Passive vs Active-Active

Active-passive may not scale well for applications with many write operations.
Active-active architecture utilizes resources very well in all the data centers.

Distributed messaging queues(Kafka and RabbitMQ)

Messaging Queue Basics:
A messaging queue involves a producer that creates messages, a queue that stores them, and a consumer that processes them.
Messaging queues offer **asynchronous processing**, **reducing latency** as the producer doesn't wait for the consumer.
They provide **retry capabilities**, ensuring messages are processed even if a consumer fails temporarily.
Messaging queues help in **pace matching**, allowing producers to send messages at a faster rate than consumers can process them.

Point-to-Point vs. Pub-Sub Messaging:
In **point-to-point**, each message in the queue is consumed and processed only once by a single consumer.
In **Pub-Sub**, a message is broadcast to all queues, allowing multiple consumers to process the same message.

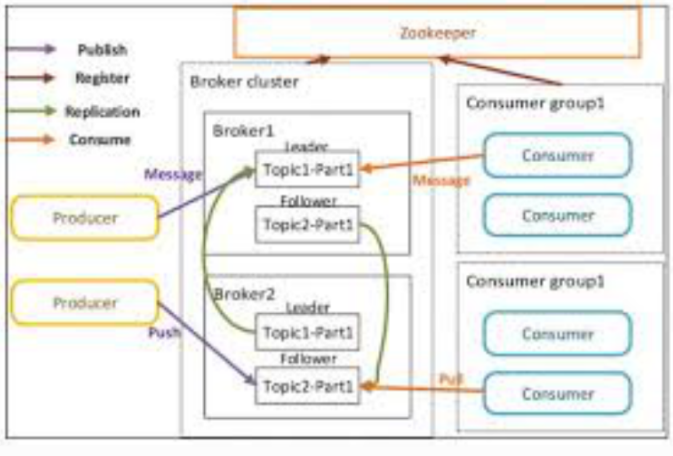
Kafka Deep Dive::

1. Key components include **producers, consumers, consumer groups, topics, partitions, offsets, brokers, clusters, and Zookeeper.**
2. **Producers** send messages to brokers, which are Kafka servers.
3. **Brokers**(servers) contain topics, and topics are divided into **partitions**.
4. **Partitions** store messages in an ordered sequence called **offsets**.
5. **Consumers** read from partitions and belong to consumer groups.
6. Consumers within the same group do not share partitions, but different groups can read the same partitions.
7. A cluster is a group of brokers, and Zookeeper helps them communicate internally.

- **Messages** in Kafka include a **key, value, partition, and topic**.
- The **key is hashed** to determine the partition, **or** messages are distributed in a **round-robin fashion**.
- Offsets track the progress of consumers, with the "committed offset" indicating the last successfully read message.
- If a consumer fails, another consumer in the group takes over from the last committed offset.
- Partitions have **leaders** and replicas (followers) for fault tolerance.
- If a consumer fails to process a message, Kafka can retry, and failed messages can be sent **to a dead-letter queue.**
- **Kafka uses a pull-based approach where consumers request messages.**

RabbitMQ Overview:

Involves **producers**, **exchanges**, **queues**, and **consumers**.
Producers send messages to exchanges, which route them to queues based on bindings and routing keys.
Different exchange types include **fanout** (broadcasts to all queues), **direct** (matches routing key exactly), and topic (uses wildcards).
If a consumer fails to process a message, RabbitMQ can requeue it.
RabbitMQ uses a **push-based approach**, sending messages to consumers as they arrive.



Proxy servers

What is a Proxy Server?
A proxy server acts as an intermediary between a client and a server. It takes requests on behalf of a client.
All client requests pass through the proxy; clients and servers do not communicate directly.

Types of Proxy Servers
Forward Proxy(Simple): Protects clients by hiding their network.
1. Hides the client network, providing anonymity.
2. Groups similar requests.
3. **Allows access to restricted** content.
4. Enhances security by controlling access.
5. **Caches** content.
6. **Works at the application level, requiring setup for each application**(Disadvantage).

Reverse Proxy: Protects servers from direct internet traffic.(**CDN**)
1. Enhances security by hiding server IP addresses.
2. Helps in handling DDOS attacks.
3. Caches content.
4. Reduces latency by serving content from locations closer to users.
5. Performs load balancing.

Proxy vs. VPN
Proxy:
- Provides anonymity, caching, and logging.
- Hides IP addresses.
- Cannot encrypt data.

VPN (Virtual Private Network):
- Creates a secure, encrypted tunnel for data transmission.
- Encrypts and decrypts data.

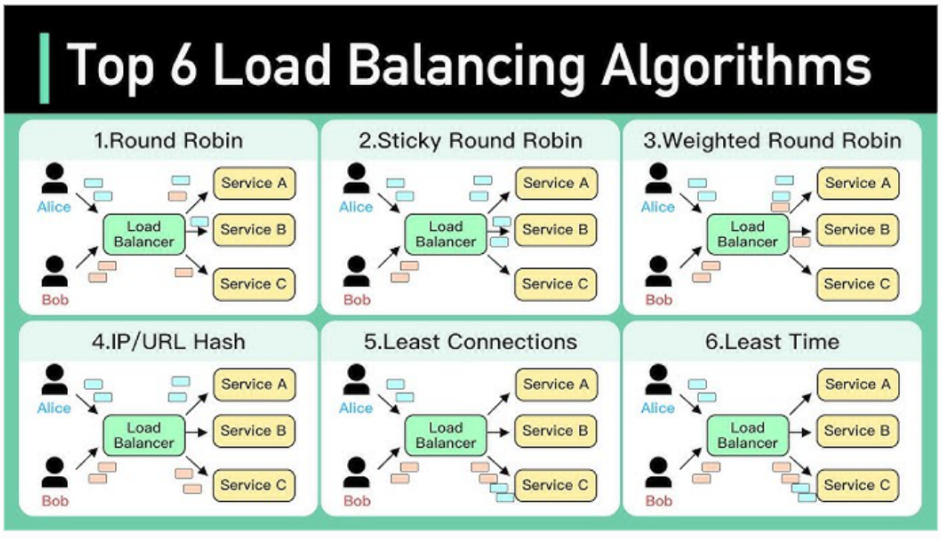
Proxy vs. Load Balancer
Reverse Proxy: Can act as a load balancer, providing IP anonymity, caching, and logging.
Load Balancer: Distributes traffic across multiple servers but does not provide the additional features of a proxy.
Not needed if there is only one server.

Proxy vs. Firewall
Firewall: Works at the packet level, scanning packet headers, port numbers, and IP addresses to enforce rules.
Proxy: Operates at the application level, with access to data for rule-based actions
Traditional firewalls work on packets, while proxy firewalls work at the application layer.

Key Terms:
Intranet: A private network accessible only to an organization's staff.
Anonymity: The state of not being identified; in this context, hiding IP addresses.
Caching: Storing frequently accessed data to serve it faster in the future.
DDOS Attack (Distributed Denial of Service): A cyber-attack where multiple systems flood the resources of a target system.
CDN (Content Delivery Network): A geographically distributed network of proxy servers and their data centers.
Latency: Delay in data transfer.
Load Balancer: Distributes network traffic across multiple servers to ensure no single server is overwhelmed.
VPN Tunnel: An encrypted connection between a device and a network.
Encryption: Converting data into a coded form to prevent unauthorized access.
Decryption: Converting encrypted data back into its original form.
Packet Scanning: Examining data packets to enforce security rules.
Application Layer: A level in network communication that focuses on the applications communicating.
Transport Layer: A level in network communication that provides reliable data transfer.
Network Layer: A level in network communication responsible for routing data packets.
Physical Layer: The lowest level in network communication, dealing with the physical connections.

Load balancers:

Load Balancer: A load balancer distributes client requests across multiple servers to prevent any single server from becoming overloaded.



Types of Load Balancers:
L4 Balancer (Network Load Balancer): Operates at the transport layer (4th layer of the OSI model). It makes routing decisions based on TCP port, UDP port, and IP addresses.
L7 Balancer (Application Load Balancer): Works at the application layer (7th layer of the OSI model). It can read headers, sessions, cookies, and data to make more informed decisions about traffic distribution. It can also perform caching.

Load Balancing Algorithms:

The algorithms are categorized into static and dynamic types.
Static Algorithms:
Round Robin: Distributes requests sequentially to each server.
Advantage: Easy to implement and ensures equal load distribution.
Disadvantage: Doesn't consider server capacity.

Weighted Round Robin: Assigns weights to servers based on their capacity. Servers with higher weights receive more requests.
Advantage: Prevents overloading low-capacity servers.
Disadvantage: Doesn't account for varying request processing times.

IP Hash: Uses the source IP address of the client to compute a hash, ensuring that the same client is always directed to the same server.
Advantage: Useful when clients need to connect to the same server consistently.
Disadvantages: If requests come through a proxy, all clients appear to have the same IP address. Also, it doesn't guarantee equal distribution.

Dynamic Algorithms:
Least Connection: Directs traffic to the server with the fewest active connections.
Advantage: Considers the current load on each server.
Disadvantage: Doesn't differentiate between connection with high or low traffic.

Weighted Least Connection: Combines the principles of least connection and weighted round robin. It considers both the server's capacity (weight) and the number of active connections.

Least Response Time: Directs traffic to the server with the fastest response time, calculated using Time to First Byte (TTFB).

TTFB (Time to First Byte): The time it takes for a client to receive the first byte of data from the server after sending a request.
Advantage: Optimizes for quick response times.

Caching strategies:

Caching: A technique to store frequently used data in a Fast Access Memory rather than accessing data every time from a slow access memory.
Fault Tolerance: Achieved through caching, specifically the "write back" strategy.

Types of Caching:

- **Client-side** (browser caching)
- **CDN** (Content Delivery Network): A geographically distributed network of proxy servers and their data centers.
- **Load balancer**: Distributes network traffic across multiple servers to ensure no single server is overwhelmed.
- **Server-side application caching** (Redis), Sits between the application server and the database.
- **Distributed Caching**: Uses a cache pool with multiple cache servers to overcome the limitations of a single cache server. (**Consistent Hashing**: Used to allot a cache server to a particular app server).

Caching Strategies
Five caching strategies using sequence diagrams:

1. **Cache-Aside:**
   - The application first checks the cache. On a **cache hit,** data is returned to the client.
   - On a **cache miss,** the application fetches data from the DB, stores it in the cache, and returns it to the client.
   - **Good for read-heavy applications**.
   - If the cache goes down, requests can still be fulfilled from the DB.
   - The cache data structure can be independent of the DB structure.
   - **Potential for inconsistency between cache and DB during write operations**.

2. **Read-Through Cache**:
   - Similar to cache-aside, but **the cache itself is responsible for fetching data from the D**B on a cache miss.
   - Good for read-heavy applications.
   - The logic for fetching and updating the cache is separated from the application.
   - C**ache document structure should be the same as the DB table.**

3.**Write-Around Cache:**
   - Data is written directly to the DB, bypassing the cache.
   - The cache is invalidated or marked as dirty.
   - Used with read-heavy applications and either read-through or cache-aside strategies.
   - Resolves inconsistency problems between cache and DB.
   - If the DB is down, write operations will fail.

4.**Write-Through Cache**:
   - Data is first written to the cache and then synchronously to the DB.
   - Cache and DB always remain consistent.
   - Increases the chances of cache hits.
   - Requires **a two-phase commit to ensure both cache and DB operations succeed or fail together**.
   - Not fully fault-tolerant; if either DB or cache goes down, the write operation will fail.

5.Write-Back (or Behind) Cache:
   - Data is first written to the cache and then asynchronously to the DB.
   - Improves fault tolerance as the system can still operate even if the DB is temporarily down.
   - Good for write-heavy applications.
   - **Provides better performance when used with read-through or cache-aside** strategies.
   - Potential issues if the time-to-live for data in the cache expires before the data is written to the DB.

Conclusion:
The video provides a comprehensive overview of caching strategies, detailing their pros and cons. It emphasizes the importance of choosing the right strategy based on the application's specific needs, particularly whether it is read-heavy or write-heavy.

Top 5 Caching Strategies

blog.bytebytego.com

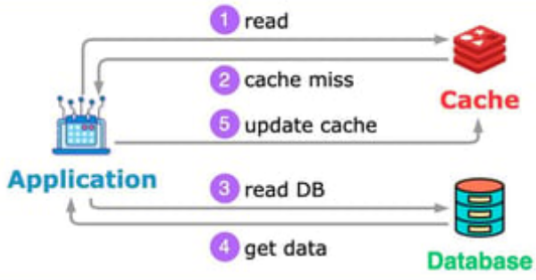
Cache Aside

Pros

1. update logic is on application level, easy to implement
2. cache only contains what the application requests for

Cons

1. each cache miss results in 3 trips
2. data may be stale if DB is updated directly



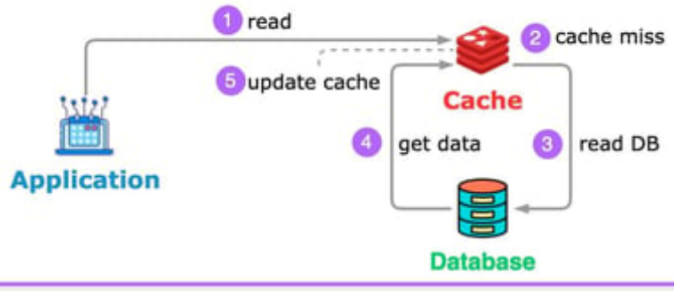
Read Through

Pros

1. application logic is simple
2. can easily scale the reads and only one query hits the DB

Cons

data access logic is in the cache, needs to write a plugin to access DB



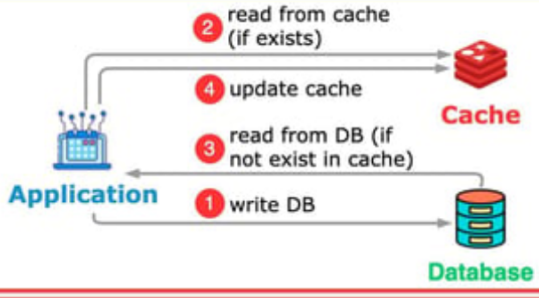
Write Around

Pros

1. the DB is the source of truth
2. lower read latency

Cons

1. higher write latency because data is written to DB first
2. the data in the cache may be stale



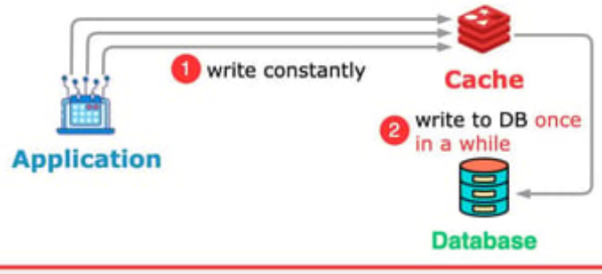
Write Back

Pros

1. lower write latency
2. lower read latency
3. the cache and DB are eventually consistent

Cons

1. there can be data loss if the cache is down
2. infrequent data is also stored in the cache



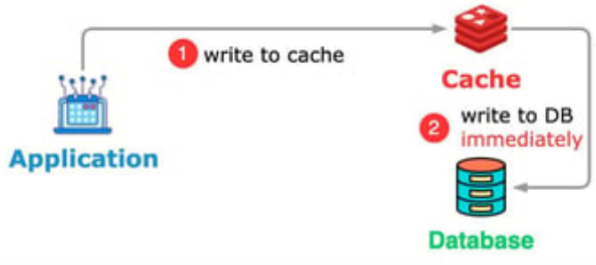
Write Through

Pros

1. reads have lower latency
2. the cache and DB are in sync

Cons

1. writes have higher latency because they need to wait for the DB writes to finish
2. infrequent data is also stored in the cache



Q/A

1. What is the primary goal of designing a high availability architecture, and what availability percentage is typically targeted?

The primary goal is to minimize downtime and ensure continuous service operation. A common target is 99.999% availability (five nines), which translates to approximately 5.26 minutes of downtime per year.

2. Explain the key differences between active-passive and active-active architectures.

Active-Passive:
One primary node/data center handles all active traffic.
Secondary nodes/data centers act as backups for disaster recovery.
Failover is required to switch to a backup.

Active-Active:
Multiple nodes/data centers handle active traffic simultaneously.
Traffic is distributed across all active nodes.
Requires complex synchronization to maintain data consistency.

3. What are the advantages and disadvantages of using an active-passive architecture?

Advantages:
Simpler to implement compared to active-active.
Lower risk of data conflicts.
Disadvantages:
Underutilization of backup resources.
Downtime during failover.
Latency can be increased if traffic needs to be routed to the primary node.

4. **What are the advantages and disadvantages of using an active-active architecture**?

Advantages:
Full utilization of all resources.
Improved performance and scalability.
Faster response times due to traffic distribution.
Disadvantages:
Increased complexity in synchronization and conflict resolution.
Higher implementation and maintenance costs.

5. How does a single point of failure impact system reliability, and how can it be avoided?

A single point of failure (SPOF) means that if one component fails, the entire system fails.
It can be avoided through redundancy:

- Duplicating critical components.
- Implementing failover mechanisms.
- Distributing workloads across multiple servers.

6. **Explain the fundamental components and workflow of a messaging queue**.

Components:

1. Producer: Creates and sends messages.
2. Queue: Stores messages.
3. Consumer: Receives and processes messages.

Workflow:

- The producer sends a message to the queue.
- The queue stores the message.
- The consumer retrieves and processes the message.

7.**What is the difference between point-to-point and publish-subscribe messaging?**
Point-to-Point:
Each message is delivered to only one consumer.
Used for tasks that need to be processed once.
Publish-Subscribe (Pub-Sub):
Each message is broadcast to all subscribers.
Used for tasks that need to be processed by multiple consumers.

8.**Describe the key components and functions of Kafka**.
Components:
1. Producers: Send messages to Kafka.
2. Brokers: Kafka servers that store messages.
3. Topics: Categories of messages.
4. Partitions: Divisions of topics for parallelism.
5. Consumers: Read messages from Kafka.
6. Zookeeper: Manages the Kafka cluster.

Functions:
High-throughput message streaming.
Fault tolerance through replication.

9. Explain the role of exchanges and routing keys in RabbitMQ.

Exchanges: Receive messages from producers and route them to queues.
Routing Keys: Used by exchanges to determine which queues to send messages to Exchanges use routing keys and bindings to route messages.

10. What are the differences between Kafka's pull-based and RabbitMQ's push-based message consumption?

Kafka (Pull-based):
Consumers request messages from brokers.
Allows consumers to control their consumption rate.
RabbitMQ (Push-based):
Brokers push messages to consumers.
Can overwhelm slow consumers.

11.What is the primary function of a proxy server, and how does it act as an intermediary?

A proxy server acts as an intermediary between clients and servers.
It receives client requests and forwards them to servers, or vice versa, masking the client's or server's identity.

12.Explain the differences between forward and reverse proxy servers.

Forward Proxy:
Protects client identities.
Used by clients to access external resources.
Reverse Proxy:
Protects server identities.
Used to distribute traffic to backend servers.

13.**How does a proxy server differ from a VPN, a load balancer, and a firewall?**

Proxy: Intermediary, provides caching and some security.
VPN: Encrypts traffic, provides secure connections.
Load Balancer: Distributes traffic across multiple servers.
Firewall: Filters network traffic based on security rules.

14. **What are the benefits of using a CDN in conjunction with proxy servers?**

CDN (Content Delivery Network):
Caches content in geographically distributed proxy servers.
Reduces latency by serving content closer to users.
Reduces load on origin servers.

15.What is the purpose of a load balancer, and why is it essential for distributed systems?

A load balancer distributes network traffic across multiple servers.
It prevents overloading any single server, improving performance and availability.

16.Explain the differences between L4 and L7 load balancers.

L4 (Network Load Balancer):
Operates at the transport layer.
Routes traffic based on IP addresses and ports.
Faster.
L7 (Application Load Balancer):
Operates at the application layer.
Routes traffic based on application data (e.g., headers, URLs).
More flexible.

17.**Describe the various static and dynamic load balancing algorithms, including Round Robin, Least Connection, and Least Response Time.**

Static:
Round Robin: Distributes requests sequentially.
Weighted Round Robin: Assigns weights to servers.
IP Hash: Routes requests based on client IP.
Dynamic:
Least Connection: Routes requests to the server with the fewest connections.
Least Response Time: Routes requests to the server with the fastest response time.

18.**What is caching, and why is it used in software architecture?**
Caching is storing frequently accessed data in fast access memory.
It reduces latency and improves performance by avoiding repeated data retrieval from slower storage.

19.**Explain the differences between cache-aside, read-through, write-around, write-through, and write-back caching strategies.**

Cache-Aside: Application manages cache reads and writes.
Read-Through: Cache fetches data from the database on a miss.
Write-Around: Writes data directly to the database, bypassing the cache.
Write-Through: Writes data to both cache and database synchronously.
Write-Back: Writes data to cache, then asynchronously to the database.

20.What is distributed caching, and why is consistent hashing used in its implementation?

Distributed Caching: Uses multiple cache servers to improve scalability and availability.
Consistent Hashing: Distributes data across cache servers, minimizing data redistribution when servers are added or removed.

Distributed transaction handling

A transaction is a set of operations performed against a database. It can be considered a group of tasks.
Example: Transferring money from account A to account B.

ACID Properties of Transactions
Atomicity: All operations in a transaction must succeed, or all must fail. If one operation fails, all changes are rolled back.
Consistency: The database should be in a consistent state before and after the transaction .
Isolation: Concurrent transactions should appear to run in a serialized manner. Locks are used to ensure operations happen sequentially.
Durability: After a transaction is successfully completed, the data should persist even if the database fails.

Handling Transactions in a Single Database
When a transaction begins, it is created on a specific database .
Operations within the transaction acquire locks on the affected rows .
On success, changes are committed, and locks are released.
On failure, changes are rolled back
Transactions are local to a particular database.

When operations involve multiple databases, a single transaction cannot ensure ACID properties across all databases
Example: Updating order and inventory databases in an e-commerce system. If updating one database fails, the other might have already been committed
Ways to Handle Distributed Transactions
1. Two-Phase Commit (2PC)
2. Three-Phase Commit (3PC)
3. Saga Pattern

Two-Phase Commit (2PC)
Involves a transaction coordinator and participants (databases/microservices).
Phase 1 (Prepare/Voting Phase): The coordinator asks all participants if they are prepared to commit. Participants put a lock and make changes but do not commit. Participants respond with "yes" or "no".

Phase 2 (Decision/Commit Phase): If all participants are prepared, the coordinator sends a commit message. If any participant is not prepared, the coordinator sends an abort message .

Failure Handling:
Maintains a log file
Prepare Message Lost: Participant aborts the transaction
OK Message Lost: Coordinator aborts the transaction after a timeout
Commit Message Lost: Participant waits for the coordinator to come back up This is a blocking phase

Three-Phase Commit (3PC)
An improvement over 2PC to address the blocking issue.
Phase 1 (Prepare Phase): Same as 2PC.
Phase 2 (Pre-Commit Phase): The coordinator sends a **pre-commit messag**e to all participants, indicating the decision to commit or abort.
Phase 3 (Commit Phase): Participants execute the commit or abort.

Non-Blocking: Participants can make decisions even if the coordinator fails.
If the pre-commit message is lost, participants can communicate with each other to decide whether to abort.

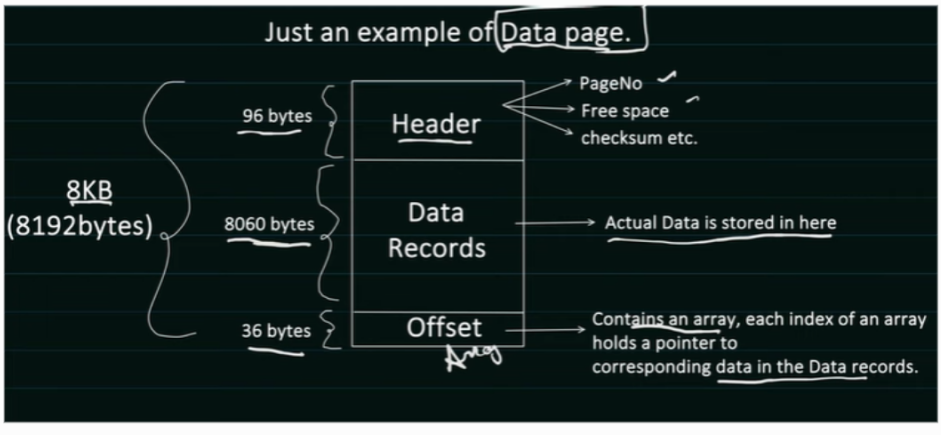
Saga Pattern
Asynchronous and used for long-trailing transactions.
Involves multiple participants in a sequential manner.
Each participant performs its local transaction and then triggers the next participant.
If one participant fails, it triggers compensating transactions in the previous participants to roll back the changes.
Uses events to trigger subsequent transactions or compensations

Comparison
2PC and 3PC are synchronous and Saga is asynchronous.
2PC and 3PC are not suitable for long-trailing activities due to locking whereas Saga is suitable for long-trailing activities where eventual consistency is acceptable

Database indexing

How Table Data is Stored:

- Tables are logically represented with rows and columns.
- DBMS creates d**ata pages (typically 8 K**B) to store table rows.
- Data pages have **a header (96 byte**s) with metadata like page number and free space.
- **Data records (8060 bytes)** within the data page store the actual rows.
- An offset array (36 bytes) contains pointers to the corresponding data rows within the data records.
- DBMS manages data pages, and **a table's data can span multiple data pages.**
- Data pages are stored in data blocks(4KB to 32 KB), which are the minimum unit of data that can be read or written in a single I/O operation.
- **DBMS maintains a mapping of data pages to data blocks**



Types of Indexing

Indexing increases the performance of database queries.
Without indexing, DBMS would have to iterate through each table row to find data, resulting in **a time complexity of O(n)**.
B+ tree data structure is commonly used for indexing, providing a time complexity of O(log n) for **insertion, searching, and deletion.**
B-trees maintain sorted data, with all leaves at the same level.
An m-order B-tree means each node can have at most m children.
Each node in a B-tree can have m-1 keys.
Leaf nodes in a B+ tree hold the indexed column values.
Root and intermediate nodes help with faster searching.

DBMS uses B+ trees to manage data pages and rows within those pages.

DBMS and B+ Trees

When a column is indexed, DBMS applies a B+ tree to manage the data.
Leaf nodes of the B+ tree hold the index column values.
Each key in the B+ tree also holds a pointer to the data page where the corresponding row is stored.
When inserting a new row, DBMS first determines the logical position in the B+ tree.
It then tries to insert the data into the data page of the nearest index value.
If the data page is full, page splitting occurs, and a new page is created.
DBMS maintains mappings between pages and data blocks.

Clustered vs. Non-Clustered Indexing

Clustered indexing: The order of rows inside the data page matches the order of the index.
Achieved using the offset array to maintain the indexing sequence.
There can be only one clustered index per table.
Primary key columns are often used as clustered keys.
If no primary key is defined, DBMS may create an internal auto-increment column for clustered indexing.
Non-clustered indexing: Creates a separate index structure.
Can have multiple non-clustered indexes per table.
Secondary indexes and composite indexes are non-clustered.
Non-clustered indexes point to the data pages where the actual data resides.

Overhead of Indexing

Each secondary index requires maintaining a B+ tree, which consumes memory.
DBMS also maintains index pages, which are stored on disk.
Adding or deleting rows requires updating both clustered and non-clustered indexes.
Page splits can occur, requiring updates to index pointers.

Steps for Searching Data

1. Load the index pages into memory.
2. Use the B+ tree to find the data page containing the required data.
3. Use the data page to find the corresponding data block.
4. Load the data block into memory.
5. Read the data.

Concurrency

Concurrency Problem: Multiple concurrent requests trying to access and modify the same resource.
Critical Section: The part of the code that accesses a **shared resource**.

Synchronized Block: Works for concurrency within **a single process** but is insufficient for distributed systems.
Distributed Concurrency Control: Needed for systems with multiple processes.

Prerequisites:
Transactions: Ensuring data **integrity** by using rollback to revert changes in case of failure.
DB Locking: **Preventing conflicting** updates using **shared** (read,S) and **exclusive** (write,X) locks.
Isolation Levels: Defining the degree of concurrency allowed, including read uncommitted, read committed, repeatable read, and serializable. Problems solved by each isolation level: **dirty reads**, **non-repeatable reads**, and **phantom reads**.

Isolation Level	Dirty Read Possible	Non-Repeatable Read Possible	Phantom Read Possible	
Read Uncommitted	Yes	Yes	Yes	*Consistency High*
Read Committed	No	Yes	Yes	
Repeatable Read	No	No	Yes	
Serializable	No	No	No	*Consistency Low*

ISOLATION LEVEL	Locking Strategy
Read Uncommitted	**Read** : No Lock acquired **Write**: No Lock acquired
Read Committed	**Read**: Shared Lock acquired and Released as soon as Read is done **Write**: Exclusive Lock acquired and keep till the end of the transaction
Repeatable Read	**Read**: Shared Lock acquired and Released only at the end of the Transaction **Write**: Exclusive Lock acquired and Released only at the end of the Transaction
Serializable	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the Transaction.

Optimistic Concurrency Control: Assumes conflicts are rare. Uses versioning to detect concurrent modifications and roll back transactions if conflicts occur. It typically uses the "read committed" isolation level.
Pessimistic Concurrency Control: Assumes conflicts are likely. Uses locking to prevent concurrent modifications. It typically uses "repeatable read" or "serializable" isolation levels. This can lead to deadlocks.
Two-Phase Locking: A popular protocol for pessimistic concurrency control.
Conclusion: The choice between optimis tic and pessimistic concurrency control depends on the specific problem and the required isolation level.

Two-Phase Locking Overview:

It is a type of pessimistic locking.
It has two phases: a growing phase and a shrinking phase.
During the growing phase, transactions acquire locks.
During the shrinking phase, transactions release locks.

Basic Two-Phase Locking:

Transactions can release locks even before committing or ending.
It can lead to deadlocks.
It can cause cascading aborts.

Deadlock Prevention Strategies:
Timeout: Aborting transactions that wait too long for a lock.
Wait-For Graph (**WFG**): Detects deadlocks by checking for cycles in a graph representing transaction dependencies.
Conservative 2PL: Transactions acquire all necessary locks at the start, preventing deadlocks but reducing concurrency.
Timestamp-Based Deadlock Detection: Uses transaction timestamps to prioritize transactions in wait-die or wound-wait schemes.

Conservative Two-Phase Locking:
Also known as static two-phase locking.
Requires transactions to acquire all locks at the start.
Avoids deadlock.
It results in less concurrency.

Strong Strict Two-Phase Locking:
Also known as rigorous two-phase locking.
Transactions hold locks until the end (commit or abort).
Resolves cascading aborts.
Deadlock is still possible.
It is widely used.

Comparison of Locking Types:

Basic 2PL can have deadlocks and cascading aborts.
Conservative 2PL avoids deadlocks but can cause cascading issues.
Strong strict 2PL prevents cascading aborts but deadlocks can still occur.

OAuth 2.0 is an authorization framework that enables secure third-party access to user-protected data.
It allows users to grant limited access to their resources on one site to another site without sharing their credentials.

Key Roles/Actors in OAuth 2.0:
Resource Owner: The user who owns the protected data (e.g., a Gmail account holder).
Client: The application requesting access to the user's data (e.g., Instagram).
Authorization Server: Issues access tokens to the client after successful authorization (e.g., Gmail authorization server).
Resource Server: Hosts the protected resources (e.g., Gmail server).

Authorization Code Grant Flow:
Registration: The client (Instagram) registers with the authorization server (Gmail), **receiving a client ID and secret.**
Authorization Request: The user attempts to log in to the client (Instagram) using their account on the resource server (Gmail). The client redirects the user to the authorization server.
A**uthentication and Consent**: The user authenticates with the authorization server (Gmail) and grants consent to share specific data with the client (Instagram).
A**uthorization Code**: The authorization server redirects the user back to the client (Instagram) with an authorization code.
Access Token Request: The client (Instagram) sends the authorization code, client ID, and client secret to the authorization server (Gmail) to request an access token.
Access Token Response: The authorization server (Gmail) issues an access token and a refresh token to the client (Instagram).
Protected Resource Request: The client (Instagram) uses the access token to request protected data from the resource server (Gmail).
Resource Response: The resource server (Gmail) validates the access token with the authorization server and responds with the requested data.

API Requests and Responses (Authorization Code Grant):

1. Registration:
Request: POST /register with client name and redirect URIs.
Response: Client ID and client secret.

2.Authorize:
Request: GET /authorize with *response type (code), client ID, redirect URI, scope, and state*.
Response: Redirect to the client's redirect URI with authorization code and state.

3.Token:

Request: POST /token with grant type (authorization_code), code, redirect URI, client ID, and client secret.
Response: Access token, token type (Bearer), refresh token, and expiration time.

Refresh Token:

Request: POST /token with grant type (refresh_token), refresh token, client ID, and client secret.
Response: New access token and refresh token.

Importance of State Parameter
The state parameter is used to prevent Cross-Site Request Forgery (CSRF) attacks.
It's a random value set by the client in the authorization request and verified in the response to ensure the request was made by the client.
Other Grant Types
Implicit Grant:

The client receives the access token directly without an authorization code.
Response type is token.
No refresh token is provided.

Resource Owner Password Credentials Grant:

The client obtains the access token by providing the resource owner's username and password.
Grant type is password.

Client Credentials Grant:

Used when the client is also the resource owner.
Grant type is client_credentials.
No refresh token is required.

I hope this structured summary provides a clear and comprehensive understanding of OAuth 2.0, its flows, and the various grant types.

Cryptography Importance: Cryptography is essential for securing daily online activities, including secure chat applications, HTTPS websites, and JWT authentication. It forms a solid foundation for system design.
Encryption: The process of converting readable data into an unreadable format, known as ciphertext. It uses a cryptographic key.
Decryption: The reverse process of converting ciphertext back into readable data, using a cryptographic key.

Symmetric Encryption: Uses the same key for both encryption and decryption.

Algorithms:
DES (Data Encryption Standard): An older algorithm with a 56-bit key, now considered insecure due to its vulnerability to brute-force attacks.
AES (Advanced Encryption Standard): A more advanced and widely used algorithm with key lengths of 128, 192, or 256 bits. AES processes data in 128-bit chunks.

Asymmetric Encryption: Employs two different keys: a private key and a public key.

Algorithms:

RSA: A popular algorithm with a 2048-bit key length, offering high security.
Diffie-Hellman: Used for secure key exchange.
DSA (Digital Signature Algorithm)
ECDH (Elliptic Curve Diffie-Hellman)

Symmetric vs. Asymmetric Encryption

Symmetric Encryption:

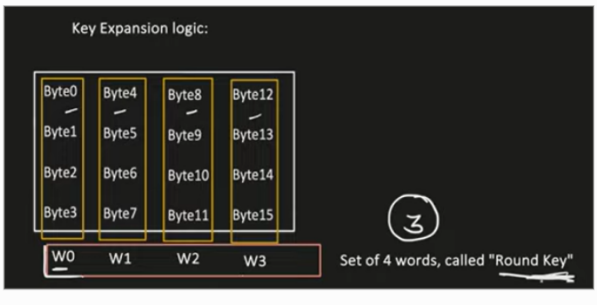
Advantage: Faster and less computationally intensive, making it suitable for encrypting bulk data.
Disadvantages: Key distribution and management become challenging as the number of clients increases.

Asymmetric Encryption:

Advantages: Eliminates key distribution security issues and enables key exchange protocols like Diffie-Hellman and digital signatures.
Disadvantage: Computationally intensive and slower, making it less suitable for encrypting large amounts of data.

AES Algorithm Details

Block Cipher: AES processes data in 1**28-bit blocks.**
State Array: The 128-bit data or key is arranged into a 4x4 matrix.
Word: A set of four bytes within the state array.
R**ound Key:** A group of four words.



Key Expansion: The process of generating 44 words from the initial 128-bit key.
Encryption Rounds: AES performs multiple rounds of operations, including Substitute Bytes, Shift Rows, Mix Columns, and Add Round Key. The number of rounds depends on the key size.

Diffie-Hellman Key Exchange

A method for securely exchanging cryptographic keys over an insecure network. It involves sharing a prime number and a primitive root, generating private and public keys, and calculating a shared secret key.

Digital Signature

Used for authentication and ensuring data integrity. The sender signs the data with their private key, and the receiver verifies the signature using the sender's public key.
Hash Function: Used to generate a fixed-size hash of the data.
Sign Algorithm: The sender's private key is used to sign the hash.
Verify Algorithm: The receiver uses the sender's public key to verify the signature.

What is **JWT?** JWT (JSON Web Token) is a secure method for transmitting information as a JSON object, digitally signed for verification, used for **authentication, authorization, and single sign-on (SSO)**.

JWT vs. Session ID:
Session ID: Requires the server to store session information in a database (**stateful**).
JWT: **Stateless**, containing all necessary information within the token itself.

JWT Structure:

Header: Metadata about the token, including type (JWT) and signing algorithm.
Payload: Contains claims or information:

Registered claims: Predefined claims like issuer (ISS), subject (SUB), audience (AUD), expiration time (EXP).
Public claims: Custom claims understood by multiple parties.
Private claims: Custom claims for internal use.

Signature: Encoded header and payload(BASE 64), signed using a cryptographic algorithm.

JWT Advantages:

Compact.
Stateless.
Digitally Signed.
Built-in Expiry.
Custom Claims.

JWT Challenges and Handling:

Token Invalidation: Difficult to invalidate before expiry. Solutions include blacklisting, changing the secret key, and using short-lived tokens.
Encoded, Not Encrypted: Base64 encoded, use JWE for more security.
Unsecured JWT: Reject tokens with "alg": "none".
JWK (JSON Web Key) Exploit: Use trusted sources for public keys.

Key Term Definitions:
JWT (JSON Web Token): An open industry
standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed.1
Authentication: The process of verifying the identity of a user or service. It answers the question "Who are you?". JWTs are often used to pass the identity of an authenticated user between different parts of an application or between applications.
Authorization: The process of determining what actions an authenticated user or service is allowed to perform. It answers the question "What are you allowed to do?". Information about a user's roles and permissions can be included in the JWT payload.

Single Sign-On (SSO):

A session and user authentication service that permits a user to use one set of login credentials (e.g.,2
username and password) to access multiple applications3 and services. JWTs can be used to propagate a user's authentication status across different applications after they log in once.
Session ID: A unique identifier generated by a server to identify a user's session. The server typically stores session-related information associated with this ID. Unlike JWTs, session IDs require the server to maintain state.
Stateless: In the context of JWTs, it means that the server does not need to store any information about the user's session. All the necessary information is contained within the token itself. This makes applications easier to scale.
Header (JWT): The first part of a JWT, which is a JSON object containing metadata about the token, such as its type (which is typically "JWT") and the signing algorithm being used (e.g., HMAC SHA256, RSA). This JSON object is then Base64 URL encoded.
Payload (JWT): The second part of a JWT, which is a JSON object containing
claims. Claims are statements about an entity (typically the user) and additional data. There are three types of claims: registered, public, and4 private. This JSON object is also Base64 URL encoded.
Claims (JWT): Statements about an entity (usually the user) and other metadata. They are key-value pairs included in the JWT payload.

Registered Claims: A set of predefined claim names that are not mandatory but are recommended to provide interoperability. Examples include iss (issuer), sub (subject), aud (audience), and exp (expiration time).
Public Claims: Claim names that can be defined by those using JWTs but should be collision-resistant. It's recommended to use a URI-like name to avoid clashes.
Private Claims: Custom claim names defined to share information between parties that agree on their meaning.

Signature (JWT): The third part of a JWT, which is used to verify that the token hasn't been tampered with and, in some cases, to ensure that the sender of the JWT is who it says it is. The signature is calculated by taking the Base64 URL encoded header and payload, a secret key (for HMAC algorithms) or a private key (for RSA or ECDSA algorithms), the specified algorithm in the header, and signing them. The result is then Base64 URL encoded.
Base64 URL Encoding: A URL-safe encoding scheme that converts binary data into an ASCII string format. JWT headers and payloads are encoded using this method. Note that encoding is not the same as encryption.
JWE (JSON Web Encryption): A standard for encrypting the contents of a JSON Web Token. While JWTs are digitally signed to ensure integrity, JWE provides confidentiality by encrypting the token's payload.
JWK (JSON Web Key): A JSON object that represents a set of public keys. JWKs are often used to distribute the public keys needed to verify the signatures of JWTs issued by a particular server or authorization service. An exploit can occur if the source of the JWK is not trusted and can be manipulated.
Blacklisting (JWT Invalidation): A method to invalidate specific JWTs before their natural expiration by maintaining a list of revoked tokens. When a request with a JWT is received, the server checks if the token is on the blacklist. This requires the server to maintain state.
Short-lived Tokens: JWTs with a short expiration time. This reduces the window of opportunity for an attacker to use a compromised token. Applications often use refresh tokens in conjunction with short-lived access tokens to provide a seamless user experience.
"alg": "none" Vulnerability: A security flaw where a JWT has its algorithm field in the header set to "none". This indicates that the token is not signed, allowing anyone to modify the header and payload without invalidating the signature (as there isn't one). Secure systems should reject tokens with this algorithm specified.

Here's a breakdown of the key terms and concepts discussed in the video, to help you better understand the potential issues and solutions for the ticket booking application:

Load Balancer: Distributes incoming requests across multiple instances of a service to ensure no single instance is overwhelmed.

Ticket Service Instances: Multiple copies of the ticket service, allowing for parallel processing of requests.

Threadpool Executor & Queue: Each service instance uses these to manage and limit the number of concurrent requests it handles.

Thundering Herd: A sudden surge of requests at a specific time, like when the ticket sales begin, which can overwhelm the system.

Retry: When a request fails, the system may automatically try again, which can exacerbate the thundering herd problem if not managed properly.

Latency: The time it takes for a request to be processed. High traffic can increase latency, potentially causing timeouts.

Auto Scaling: Automatically adding more service instances to handle increased traffic. However, it might not be enough to counter the effects of retries.

Exponential Backoff: A strategy to delay retries, with the delay increasing with each subsequent failure. This helps to avoid overwhelming the system. The formula is base * 2^n, where n is the number of failures.

Jitter: Introducing randomness into the backoff time to prevent all retries from happening simultaneously. The formula is min(maximum wait time, random(0, base * 2^n)).

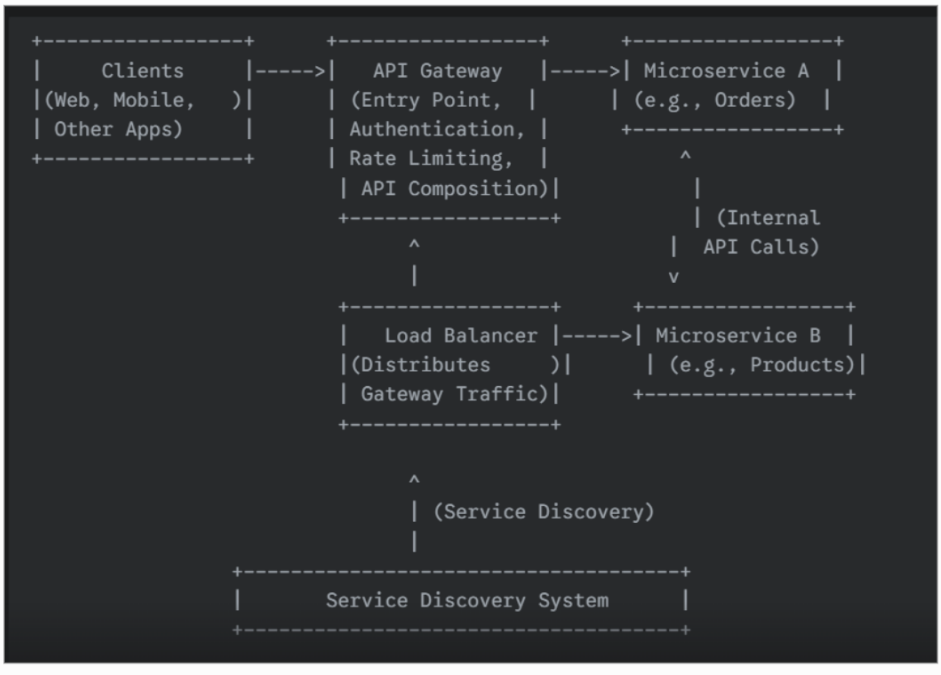
Rate Limiter: Limits the number of requests allowed over a certain period, preventing the system from being flooded.

Token Bucket Algorithm: A specific rate-limiting algorithm that controls the rate at which requests are processed.

The video suggests that the combination of rate limiting, exponential backoff with jitter, and auto-scaling can help mitigate the thundering herd effect and ensure the ticket booking application remains stable during high-demand events.

Is there anything else I can help you with?

Okay, here's a conceptual diagram illustrating the architecture discussed in the video about API Gateways in a microservices environment. Since I can't directly draw



Key Components and Concepts Illustrated:
1. **Clients**: Various applications that need to consume backend services.
2. **API Gateway**: The single entry point for all client requests. It handles authentication, rate limiting, and can compose responses from multiple microservices.
3. **Load Balancer (for API Gateways)**: Distributes traffic across multiple instances of the API Gateway for scalability and availability.
4. **Microservice A & B**: Independent backend services that perform specific business functions. They communicate internally, often through APIs.
5. **Service Discovery System**: Allows microservices (and potentially API Gateways) to locate each other dynamically.

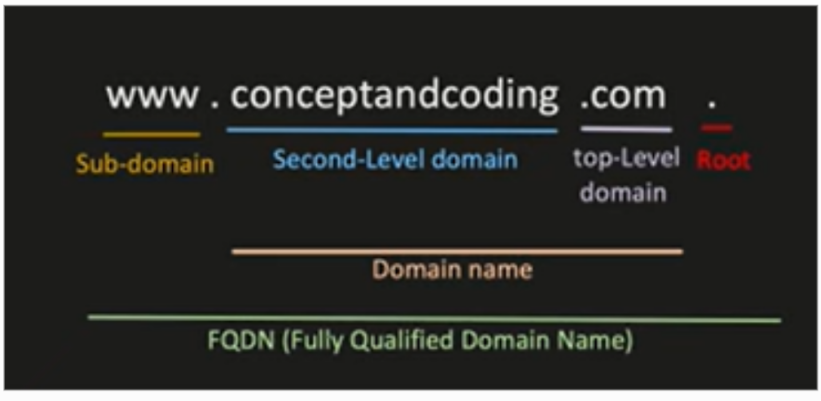
Implicit Concepts (Not Directly Visualized but Important):
* **API Composition**: The API Gateway fetching data from both Microservice A and B to provide a unified response to clients.
* **Authentication**: The API Gateway verifying the identity of the clients before routing requests.
* **Rate Limiting**: The API Gateway controlling the number of requests from clients to protect backend services.
* **Region/Availability Zones**: While not explicitly shown, the components would ideally be deployed across multiple Availability Zones within a Region for high availability. DNS-based load balancing could distribute traffic across different Regions.

Explanation of the Diagram:
Clients (web, mobile, etc.) send their requests to the API Gateway.
The API Gateway acts as the central point of contact. It handles tasks like:

Authentication: Verifying the client's identity.
Rate Limiting: Controlling the number of requests.
API Composition: Aggregating data from multiple backend services.
Routing: Directing requests to the appropriate Microservices.

A Load Balancer sits in front of multiple instances of the API Gateway to distribute traffic and ensure high availability of the gateway itself.
Microservices (A and B) are independent services that handle specific business logic. They might communicate with each other internally.
The Service Discovery System allows the API Gateway (and other microservices) to dynamically find the network locations of the backend services.

This diagram provides a high-level overview of the architecture discussed in the video. Let me know if you'd like a more detailed or specific representation of any part!



IP Address: A unique numerical label assigned to each device connected to the Internet, allowing devices to locate each other.
Domain Name: A human-readable and friendly address used to identify a device on the internet, like "google.com".
DNS (Domain Name System): A mediator that translates domain names into IP addresses, as devices communicate using IP addresses.
FQDN (Fully Qualified Domain Name): The complete domain name, including the subdomain, second-level domain, top-level domain, and root (e.g., www.conceptandcoding.com).
DNS Record: Contains information about a domain, including:

Record Name: The domain name or subdomain name.
C Name Record (Canonical Name): An alias that maps one domain to another, often used for subdomains.
A Record (Address Record): Maps a record name to an IP address.

DNS Client (Stub Resolver): A client available at the OS level that checks its local cache for DNS information.
DNS Resolver: A server that helps find the IP address of a domain. It's often provided by your ISP (Internet Service Provider).
Root Server: The top level of the DNS hierarchy. There are 13 root servers worldwide.
TLD (Top Level Domain): The last part of a domain name (e.g., .com, .org, .net).
Authoritative Name Servers: Servers that hold the actual DNS records for a domain.
DNS Zone: A way to divide the management of subdomains across multiple authoritative servers to distribute the load.
Recursive Approach: The DNS resolver handles the process of querying different servers to find the IP address.
Iterative Approach: The DNS client is responsible for making the recursive calls to resolve the address.

Is there anything else I can help you with?